



Exception Handling

Example 1: Basic try-except

```
# This code handles divide by zero error
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ZeroDivisionError:
    print("Cannot divide by zero")
# input: 0
# output: Cannot divide by zero
```

Homework:

Write a program to divide 100 by a user-input number. Handle both zero and non-integer inputs.

Example 2: Multiple Exceptions

```
# This code handles multiple types of exceptions
try:
    num = int(input("Enter an integer: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
except ValueError:
    print("Error: Invalid integer input")
# input: abc
# output: Error: Invalid integer input
```

Homework:

Add a finally block to print "Operation complete" no matter what happens.



Example 3: Else, Finally, and Raise

```
# This code shows else, finally and manual exception raising
try:
    age = int(input("Enter your age: "))
    if age < 0:
        raise ValueError("Age cannot be negative")
except ValueError as e:
    print("Error:", e)
else:
    print("Your age is", age)
finally:
    print("End of program")
# input: -5
# output: Error: Age cannot be negative
# output: End of program
```

Homework:

Catch non-integer inputs and print "Invalid input" if they enter letters.

Example 4: try-except Inside Loop for Retrying Input

```
# This code keeps asking for number until user enters valid one
while True:
    try:
        num = int(input("Enter a number: "))
        print("You entered:", num)
        break
    except ValueError:
        print("Invalid input. Try again.")
# input: abc → 25
# output:
# Invalid input. Try again.
# You entered: 25
```



Homework:

Ask user to enter a number between 1 and 10. Keep retrying if input is not a number or out of range.

Example: Custom Exception Class

```
# This code defines and raises a custom exception
class NegativeNumberError(Exception):
    pass

num = int(input("Enter a positive number: "))
if num < 0:
    raise NegativeNumberError("Negative number not allowed")
else:
    print("Number is:", num)
# input: -5
# output: Negative number not allowed (Custom error)
```

Homework:

Create a custom exception `InvalidNameError`. Raise it if user enters a blank name or number instead of text.



Modules and Packages

Example 1: Using Standard Module – math

```
# This code shows how to use the math module
import math
print(math.sqrt(16))
# input: none
# output: 4.0
```

Homework:

Ask the user for a number and print its factorial using `math.factorial()`.

Example 2: Using random Module

```
# This code generates a random number between 1 and 100
import random
num = random.randint(1, 100)
print(num)
# input: none
# output: 42 (random)
```

Homework:

Generate a random float between 0 and 1 using `random.random()`.

Example 3: Custom Module Creation and Use

File: `greeting.py`

```
def greet(name):
    print("Hello", name)
```



Main File: main.py

```
# This code uses a custom greeting module
import greeting
greeting.greet("Pushkar")
# input: none
# output: Hello Pushkar
```

Homework:

Create a new module with a function that returns the square of a number. Use it in another file.

Example 4: Importing Specific Function

```
# This code imports and uses only one function from math
from math import factorial
print(factorial(5))
# input: none
# output: 120
```

Homework:

Import ceil from math and round up 3.14. Print the result.

Example 5: Check File Exists Before Reading (using os)

```
# This code checks if file exists before reading it
import os

filename = "data.txt"

if os.path.exists(filename):
    with open(filename, "r") as file:
        content = file.read()
        print(content)
else:
    print("File not found")
```

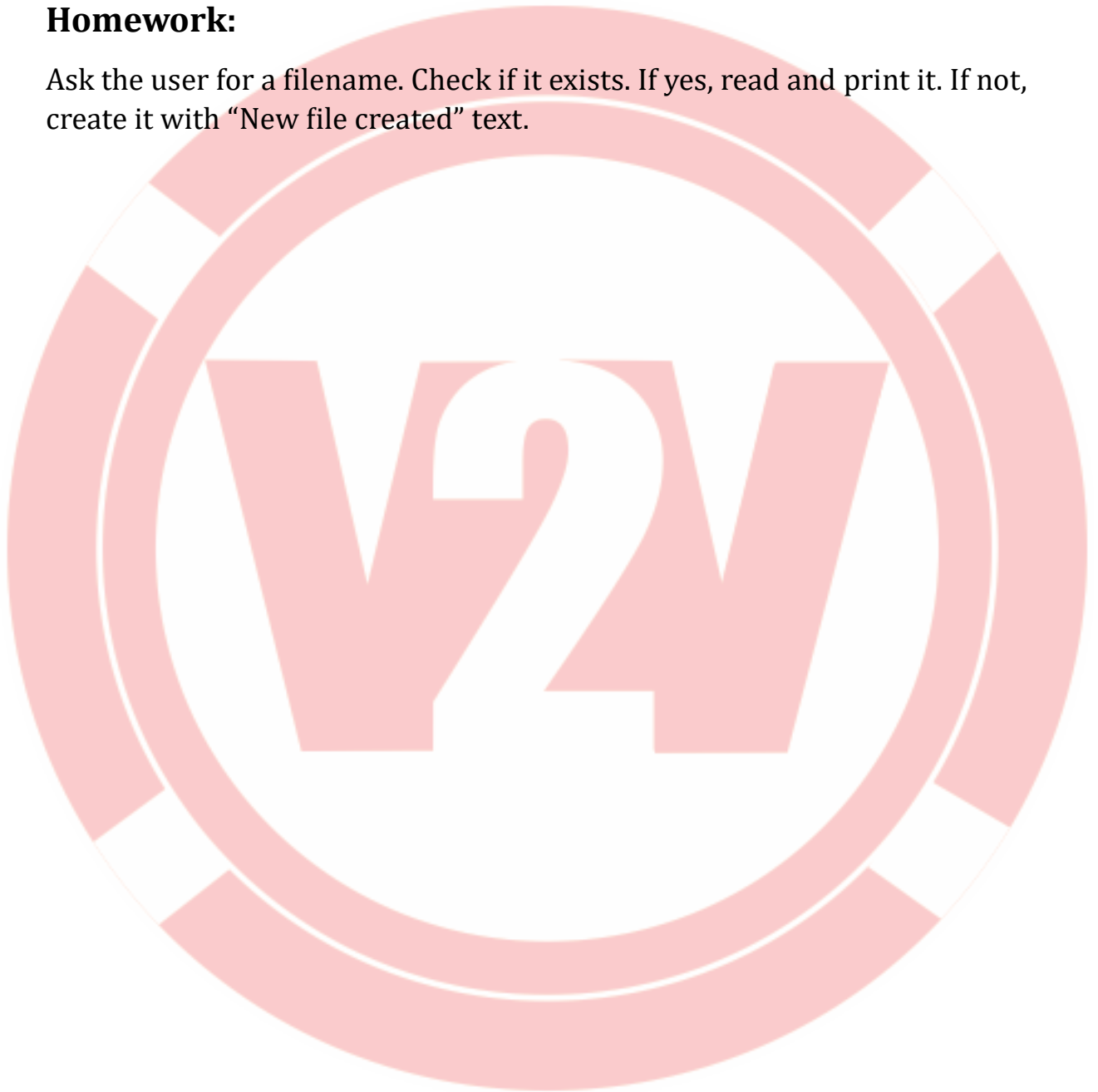


V2V EDTECH LLP
DIPLOMA | DEGREE | BSCIT

```
# input: (no file or data.txt exists)
# output: File not found OR file content
```

Homework:

Ask the user for a filename. Check if it exists. If yes, read and print it. If not, create it with "New file created" text.





File Handling

Example 1: Writing to a File

```
# This code writes text into a file
file = open("data.txt", "w")
file.write("Hello Python\n")
file.close()
print("Data written")
# input: none
# output: Data written
```

Homework:

Write "Python is fun!" into a new file called "notes.txt".

Example 2: Reading from a File

```
# This code reads the full file content
file = open("data.txt", "r")
content = file.read()
file.close()
print(content)
# input: (file contains "Hello Python")
# output: Hello Python
```

Homework:

Use a with block to read and print contents of "data.txt".

Example 3: Appending to a File

```
# This code appends a new line to a file
with open("data.txt", "a") as file:
    file.write("Appended line\n")
print("Data appended")
# input: none
```



```
# output: Data appended
```

Homework:

After appending, read and print the file to check updated contents.

Example 4: Reading File Line-by-Line

```
# This code reads and prints lines without extra newline
with open("data.txt", "r") as file:
    lines = file.readlines()
for line in lines:
    print(line.strip())
# input: (file contains two lines)
# output:
# Hello Python
# Appended line
```

Homework:

Print line numbers with each line like "1: Hello Python", "2: Appended line"